Solution Requirements

Date	06 July 2026
Project Name	**Rising Waters**
Maximum Marks	4 Marks

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority (High / Medium /Low)
1	Authentication	Users can access the flood prediction system through the web application. No login is required for basic predictions.	Medium
2	Authorization	All users have permission to enter weather parameters and view prediction results.	Medium
3	External Interfaces	The system uses the trained XGBoost model, Standard Scaler, Flask framework, HTML/CSS/JavaScript, and the flood dataset.	High
4	Transaction Processing	The system accepts weather parameters, preprocesses the data, predicts flood risk, and displays the result instantly.	High
5	Reporting	The system displays the prediction result (Flood Risk / No Flood Risk) and relevant safety information on the result page.	Medium
6	Business Rules	Prediction is generated only after all required weather parameters are entered with valid values.	High
7	Compliance to Laws or Regulations	The system stores no personal user information and is intended only for educational and research purposes.	Medium

8	Other	The web application provides a simple, responsive, and easy-to-use interface for flood prediction.	Medium

Step 2: Non Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance	The system should generate flood predictions quickly.	Prediction displayed within 3 seconds.
2	Reliability	The application should provide consistent and accurate predictions.	System availability ≥ 99% during normal use.
3	Usability	The interface should be simple and easy for any user to operate.	Users can complete a prediction in 3 or fewer steps.
4	Security	User input should be processed securely without exposing data.	Input validation enabled; no personal data stored.
5	Scalability	The system should support multiple users making predictions.	Able to handle 50+ concurrent prediction requests.
6	Maintainability	The code should be modular and easy to update.	Well-organized Flask project with separate templates, static files, and model files.
7	Compatibility	The application should work on common browsers and devices.	Supports Chrome, Edge, Firefox, and mobile browsers.
8	Availability	The application should be accessible whenever the server is running.	Available 24×7 while the server is active.